

Digital Signatures

1. Overview

A digital signature is a cryptographic mechanism that authenticates a message or document — proving it genuinely originated from a specific sender and has not been altered since signing, much like a handwritten signature on paper, but with mathematically verifiable guarantees a handwritten signature cannot offer.

Digital signatures are built on **asymmetric (public-key) cryptography** (see the RSA and ECC guides), using the sender's **private key to sign** and the corresponding **public key to verify**.

Digital Signatures at a Glance

Property	Detail
Type	Cryptographic authentication mechanism built on asymmetric key pairs
Guarantees	Authentication, integrity, non-repudiation
Common algorithms	RSA (with PSS padding), ECDSA, EdDSA (Ed25519)
Not the same as	Encryption — a signature proves origin/integrity; it does not hide content

2. What a Digital Signature Actually Proves

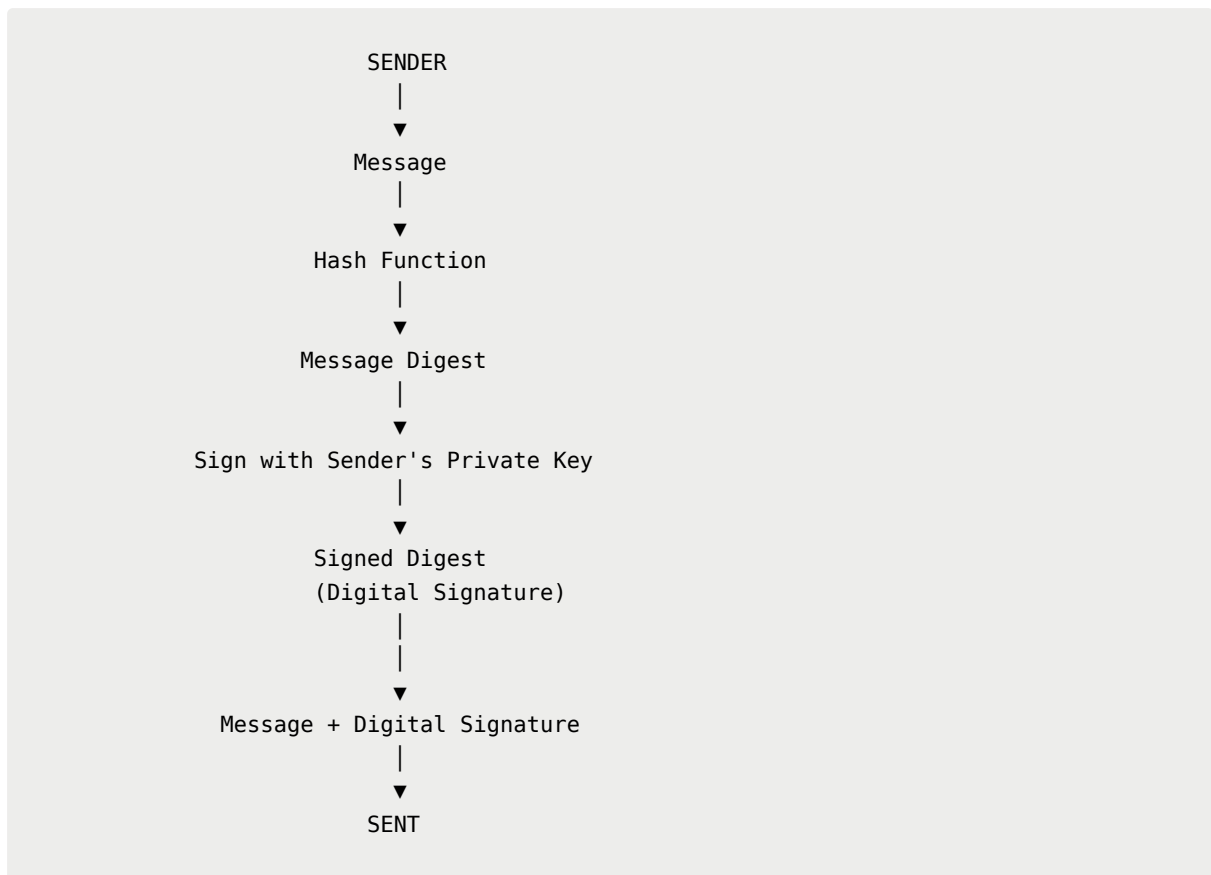
A valid digital signature gives the recipient three specific, distinct guarantees:

1. **Authentication** — the message was created by the party holding the corresponding private key.
2. **Integrity** — the message was not altered after signing; any change to the message invalidates the signature.
3. **Non-repudiation** — the signer cannot later credibly deny having signed the message, since only they possess the private key that could have produced a valid signature.

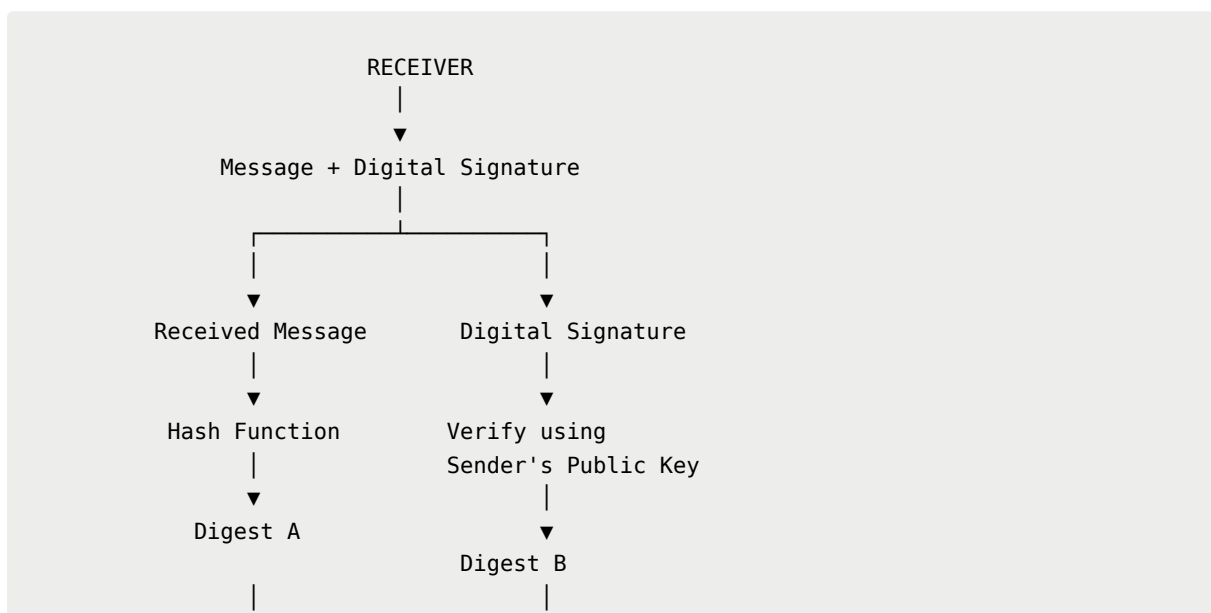
3. The Signing and Verification Process

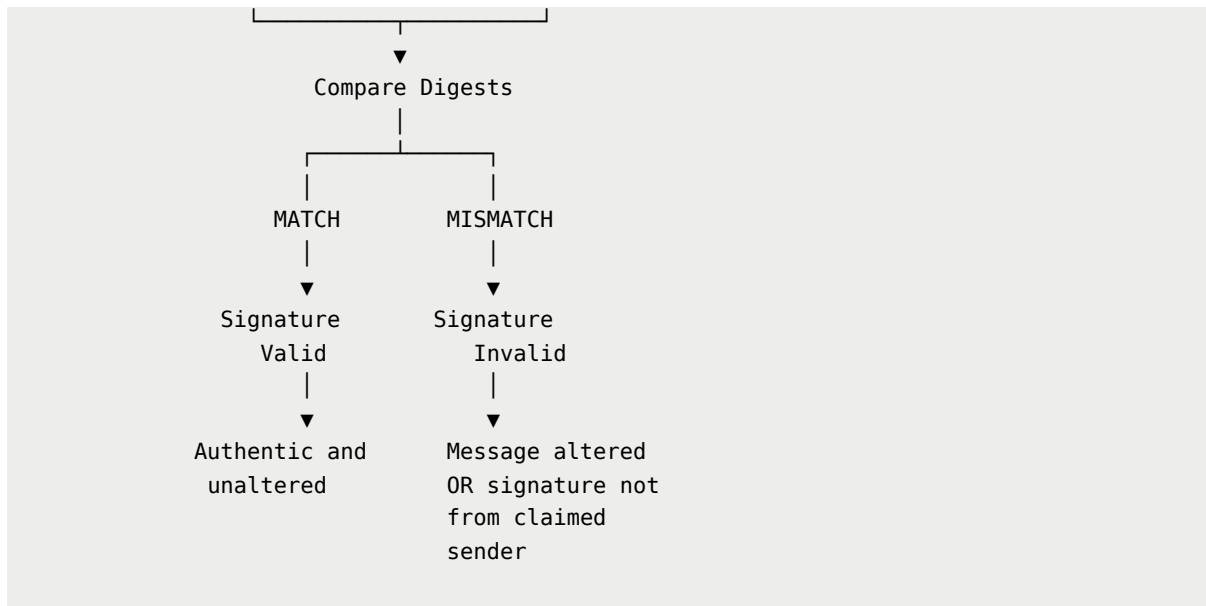
Signing a raw message directly with a slow asymmetric algorithm would be inefficient for large documents, so digital signatures are built around **hashing** (see the SHA guide) combined with asymmetric cryptography.

Signing — Sender



Verification — Receiver





Step-by-Step Verification

1. The receiver independently computes:

`Hash Function(received message) → Digest A`

2. The receiver verifies/decrypts the received signature using the **sender's public key**:

`Digital Signature → Digest B`

3. The receiver compares **Digest A** and **Digest B**.
4. **MATCH** → signature is valid; the message is authentic and unaltered.
5. **MISMATCH** → signature is invalid; the message was altered, or the signature was not produced by the claimed sender's private key.

Important: For RSA-based signatures, the original material describes the mathematical operation conceptually as "encrypting" the digest with the private key (the inverse RSA operation relative to normal encryption). ECDSA and EdDSA use structurally different signing algorithms based on elliptic-curve point operations rather than direct encryption of the digest.

The conceptual role remains the same across these algorithms: **sign with the private key, verify with the public key, against a hash of the message.**

4. Why Hashing First Matters

Efficiency

Hashing reduces an arbitrarily large document to a small, fixed-length digest before the computationally expensive asymmetric operation is applied.

This makes signing large files practical.

Security

Because a secure hash function is **collision-resistant** (see the SHA guide), it is infeasible for an attacker to find a different message producing the same digest.

Without collision resistance, an attacker could potentially substitute malicious content while preserving a "valid" signature.

Broken Hash Functions Undermine Signatures

Using a broken hash function such as **MD5** or **SHA-1** undermines the entire signature scheme, regardless of how strong the underlying asymmetric algorithm is.

A collision in the hash function can be leveraged to forge a signature over attacker-chosen content.

Key point: The security of a digital signature depends not only on the asymmetric algorithm, but also on the security of the hash function used with it.

5. Common Signature Algorithms

Algorithm	Basis	Notes
RSA-PSS	RSA with Probabilistic Signature Scheme padding	Widely deployed; PSS padding is preferred over older, less rigorously secure padding schemes.
ECDSA	Elliptic curve discrete logarithm problem	Smaller signatures than RSA at equivalent security; requires a securely generated, non-reused random nonce per signature. See the ECC guide's nonce-reuse risk.
EdDSA (Ed25519)	Edwards-curve variant	Deterministic signing — no per-signature random nonce is needed, avoiding an entire class of

implementation errors that have historically affected ECDSA.

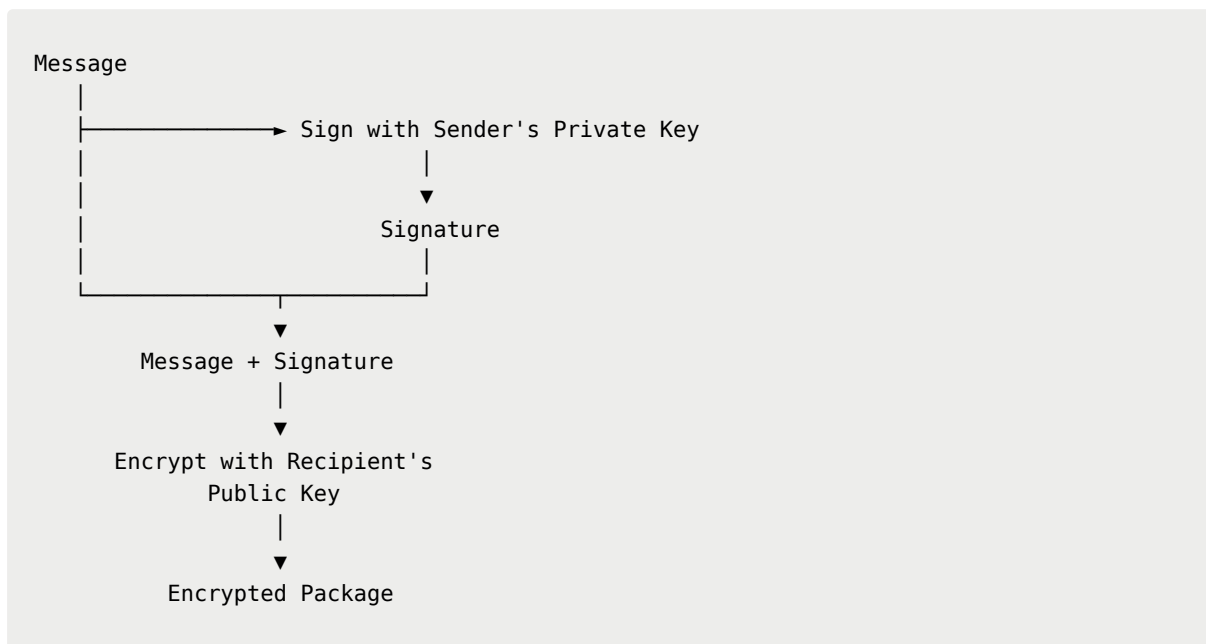
6. Digital Signatures vs. Encryption — A Common Point of Confusion

	Encryption	Digital Signature
Purpose	Confidentiality — hide content from unauthorized readers	Authentication, integrity, non-repudiation — prove origin and detect tampering
Key used to "lock"	Recipient's public key	Signer's private key
Key used to "unlock"	Recipient's private key	Signer's public key
Who can read/verify	Only the recipient (holder of the private key)	Anyone (holder of the signer's public key)

Can a Message Be Both Encrypted and Signed?

Yes.

These are independent operations and are often combined. For example:



The result provides both **authentication/integrity** through the signature and **confidentiality** through encryption.

7. Where Digital Signatures Fit Into Broader Systems

Digital signatures underpin several larger structures covered in the companion guides.

PKI and Certificates

A certificate is itself a digitally signed document.

A **Certificate Authority (CA)** signs a certificate with its own private key. That signature is what allows relying parties to trust that the certificate's claimed public key belongs to its stated subject.

TLS/HTTPS

Server certificates are validated via **signature verification** during the TLS handshake.

Some TLS handshake modes also involve the server digitally signing key-exchange parameters to prove possession of the private key matching its certificate.

8. Typical Use Cases

Digital signatures are commonly used for:

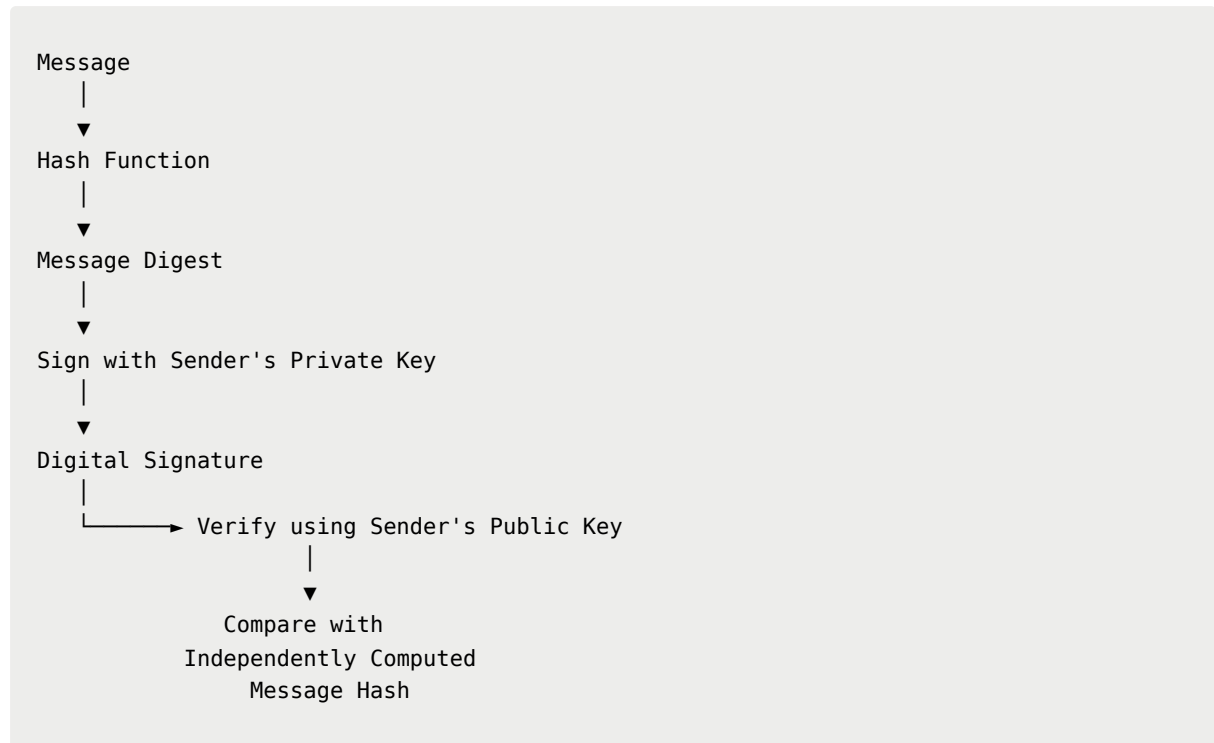
- **Code signing** — verifying software has not been tampered with since the publisher signed it.
 - **Digitally signed contracts** and legally binding e-signature systems.
 - **Secure email** — S/MIME and PGP, signing messages to prove sender authenticity.
 - **Certificate signing within a PKI** — a CA signing subordinate/leaf certificates.
 - **Software update mechanisms** — verifying update packages before installation.
 - **Blockchain transaction authorization** — transactions are signed with the owner's private key.
-

9. Summary

Digital signatures provide:

- **Authentication**
- **Integrity**
- **Non-repudiation**

They achieve this by combining **hashing with asymmetric cryptography**:



Their security depends jointly on:

1. The strength of the underlying **asymmetric algorithm**
2. The **collision resistance** of the hash function
3. Correct implementation and key management

Digital signatures form the trust foundation underlying **certificates, PKI, and secure protocols such as TLS**.

Bottom line: A digital signature proves **who signed the message and whether it was altered**; it does **not** provide confidentiality by itself. For confidentiality, encryption is required.